

Defesa - Explicação Técnica

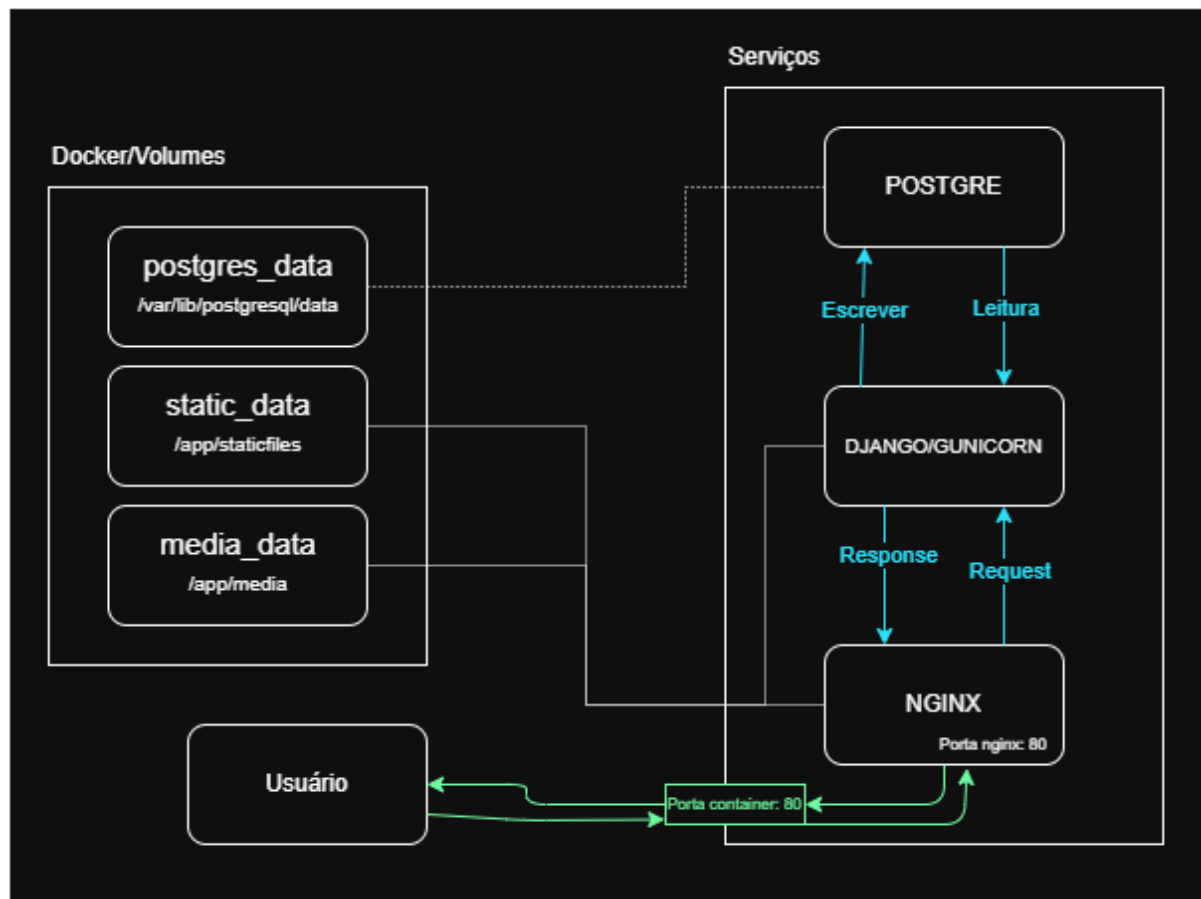
Disciplina: Computação em nuvem

Data: 26/09/2026

Docente: Fábio Castro

Discente: Antonio Paulo e Danilo Fernandes

1 - Visão geral da arquitetura



[Repositório GITHUB](#)

2 - Imagens docker da aplicação django

a) FROM python:3.11-slim

i) Por que python:3.11-slim ?

- 1) A imagem oficial padrão do Python possui mais de 1 GB, pois inclui compiladores e pacotes extras do sistema operacional. A versão slim é uma distribuição minimalista baseada em Debian com apenas ~150 MB.
- 2) Velocidade de Deploy (CI/CD): Menos dados para transferir na rede (*data egress/ingress*).
- 3) Superfície de Ataque Reduzida: Quanto menos utilitários desnecessários no container, menor o risco de vulnerabilidades ativas.

b) Variáveis de Ambiente (ENV)

i) PYTHONDONTWRITEBYTECODE=1

- 1) Impede que o Python crie arquivos compilados .pyc no sistema de arquivos. Em containers efêmeros, salvar bytecode no disco gera escrita inutilmente em camadas descartáveis.

ii) ENV PYTHONUNBUFFERED=1

- 1) Garante que o stdout e o stderr do Python sejam enviados diretamente para o terminal, sem retenção em memória (*buffer*). Isso é essencial na nuvem para que os sistemas de observabilidade (como docker logs, AWS CloudWatch ou Datadog) recebam os logs de erro e acesso em tempo real.

c) Diretório de Trabalho (WORKDIR /app)

- i) Define /app como o diretório-raiz onde todos os comandos seguintes (COPY, RUN, CMD) serão executados.
- ii) Justificativa: Organiza o sistema de arquivos do container, evitando a gravação de arquivos na raiz (/) do Linux e garantindo um contexto de execução isolado.

d) Dependências do Sistema e Limpeza de Camadas (RUN apt-get...)

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*
```

- i) O que faz: Instala suporte C (libpq-dev) para compilar o driver do PostgreSQL (psycopg2).
 - 1) Motivo: O rm -rf remove o cache de pacotes na mesma camada (*layer*), evitando que arquivos temporários inflem a imagem.

e) Estratégia de Cache por Camadas (COPY e RUN pip...)

```
# ETAPA 1: Dependências do Python
COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt

# ETAPA 2: Código-fonte da Aplicação
COPY . /app/
```

- i) O que faz: Instala as dependências *antes* de copiar o código-fonte.
 - 1) Motivo: Aproveita o Layer Cache. Se mudar apenas o código, o Docker não reinsere/reinstala as bibliotecas, reduzindo o build para segundos.

f) Exposição da Porta (EXPOSE 8000)

- i) Funciona como documentação declarativa e contrato de interface dentro do Docker, indicando que a aplicação dentro do container escuta na porta 8000. *(A liberação e o roteamento de rede reais são gerenciados pelo Docker Compose e pelo Nginx).*

g) Servidor de Aplicação WSGI e Entrada (CMD ["gunicorn"...])

```
CMD ["gunicorn", "aula_19_09.wsgi:application", "--bind", "0.0.0.0:8000"]
```

- i) Sintaxe [...]: Roda como PID 1, garantindo desligamento seguro (*graceful shutdown*) sem corromper dados.
- ii) Gunicorn: Servidor de produção multi-processo (o runserver do Django é mono-thread e trava sob carga).
- iii) 0.0.0.0:8000: Libera a escuta para a rede do Docker, permitindo que o Nginx consiga se conectar.

3 - Orquestração com Docker Compose

Serviço db

Nesse serviço de banco de dados é usada a imagem `postgres:16-alpine` que será instalada com o download via *docker hub*.

Caso o serviço pare por algum erro, esse parâmetro define o reinício do serviço `restart: unless-stopped`

Definição dos parâmetros básicos do banco de dados(nome do banco, usuário, senha):

```
environment:
  POSTGRES_DB: ${POSTGRES_DB:-appdb}
  POSTGRES_USER: ${POSTGRES_USER:-appuser}
  POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-apppassword}
```

Ponto de montagem do volume onde os dados persistem:

```
volumes:
  - postgres_data:/var/lib/postgresql/data
```

Checação de saúde do serviço:

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-appuser}
-d ${POSTGRES_DB:-appdb}"]
  interval: 5s
  timeout: 5s
  retries: 5
```

Define a rede interna que o serviço usa para se comunicar com os outros serviços:

```
networks:
  - backend
```

Serviço web

Nesse serviço roda a aplicação *django* que atua como o backend do sistema a imagem é construída com base no dockerfile dentro do repositório `./django_app`.

Caso o serviço pare por algum erro, esse parâmetro define o reinício do serviço `restart: unless-stopped`

Carrega as variáveis de ambiente:

```
env_file:
  - .env
```

Parâmetros que o django vai usar para se comunicar com o banco de dados postgresql:

```
environment:
  POSTGRES_HOST: db
  POSTGRES_PORT: 5432
```

Ponto de montagem do volume onde os dados persistem:

```
volumes:
  - media_data:/app/media
  - static_data:/app/staticfiles
```

Define a dependência da iniciação nesse serviço que é o banco de dados está ativo com a condição de saúde.

```
depends_on:
  db:
    condition: service_healthy
```

Define a rede interna que o serviço usa para se comunicar com os outros serviços:

```
networks:
  - backend
```

Serviço nginx

Nesse serviço roda a aplicação *nginx* que atua como o servidor web do sistema a imagem é construída com base no dockerfile dentro do repositório `./nginx`.

Caso o serviço pare por algum erro, esse parâmetro define o reinício do serviço `restart: unless-stopped`

Configuração das portas que serão utilizadas pelo serviço, (porta da máquina/porta do container):

```
ports:
  - "80:80"
```

Ponto de montagem do volume onde os dados persistem:

```
volumes:
  - media_data:/app/media:ro
  - static_data:/app/staticfiles:ro
```

Define a dependência da iniciação nesse serviço que é o serviço django está ativo.

```
depends_on:
  - web
```

Define a rede interna que o serviço usa para se comunicar com os outros serviços:

```
networks:
  - backend
```

4 - Comunicação entre os containers;

Todos os serviços (db, web, nginx) estão conectados à mesma rede virtual personalizada chamada backend com o driver bridge.

Quando os containers sobem na mesma rede, o Docker ativa um servidor DNS interno automático. Esse DNS mapeia os nomes dos serviços (db, web, nginx) para os IPs virtuais internos que cada container recebe dinamicamente.

- O db (Postgres) escuta na porta 5432 interna.
- O web (Django/Gunicorn) escuta na porta 8000 interna.
- Segurança: Nem o db nem o web expõem portas para a máquina hospedeira (*Host*).
- Apenas o container nginx mapeia a porta pública (ports: - "80:80"), atuando como porta de entrada (*Gateway/Ingress*).

2. Nginx → Django / Gunicorn

- Roteamento (Camada 7): O Nginx recebe a requisição na porta 80. Se for arquivo estático ou mídia, ele serve direto do volume; se for rota de sistema (/), repassa via `proxy_pass`.
- Resolução DNS: O Nginx faz a chamada para `web:8000`, e o DNS interno do Docker traduz o nome `web` para o IP privado do container Django.

3. Django → PostgreSQL

- Conexão Direta: O Django lê a variável `POSTGRES_HOST`: db e abre conexão na porta interna 5432. O DNS do Docker resolve db para o IP do container Postgres.
- Ordem de Inicialização (healthcheck): O Compose usa o teste `pg_isready` para garantir que o Django só tente conectar após o banco de dados estar 100% pronto para uso.

5 - Nginx e proxy reverso

Papel do Nginx na Arquitetura

- Porta de Entrada (Gateway): Isola o Django e o banco de dados da internet direta.
- Descarregamento de Carga (*Offloading*): Serve estáticos e mídia diretamente sem gastar CPU ou memória do Python.

Fluxo de Requisições e Portas Envolvidas

- Porta 80 (Pública): Escuta o tráfego vindo da internet/navegador.
- Porta 8000 (Privada): Porta interna do container Django para onde o Nginx repassa as requisições da aplicação.

Definição do Servidor de Aplicação (upstream)

- `upstream django { server web:8000; }`: Mapeia o destino final do tráfego.
- DNS do Docker: Traduz o nome do serviço web para o IP privado do container na rede interna.

Servimento Direto de Estáticos e Mídia

- `location /static/e/media/`: Apontam via alias diretamente para os volumes compartilhados em disco.
- Autonomia: Responde ao cliente sem acionar o código Python/Gunicorn.

A Configuração do Proxy Reverso

- `proxy_pass http://django;`: Executa o proxy reverso, repassando as rotas dinâmicas (/) para o servidor WSGI-Gunicorn no trabalho.
- `proxy_set_header`: Preserva o IP real e o Host do cliente para permitir as validações de segurança do Django (`CSRF_TRUSTED_ORIGINS`).

Trechos Principais do `nginx.conf`

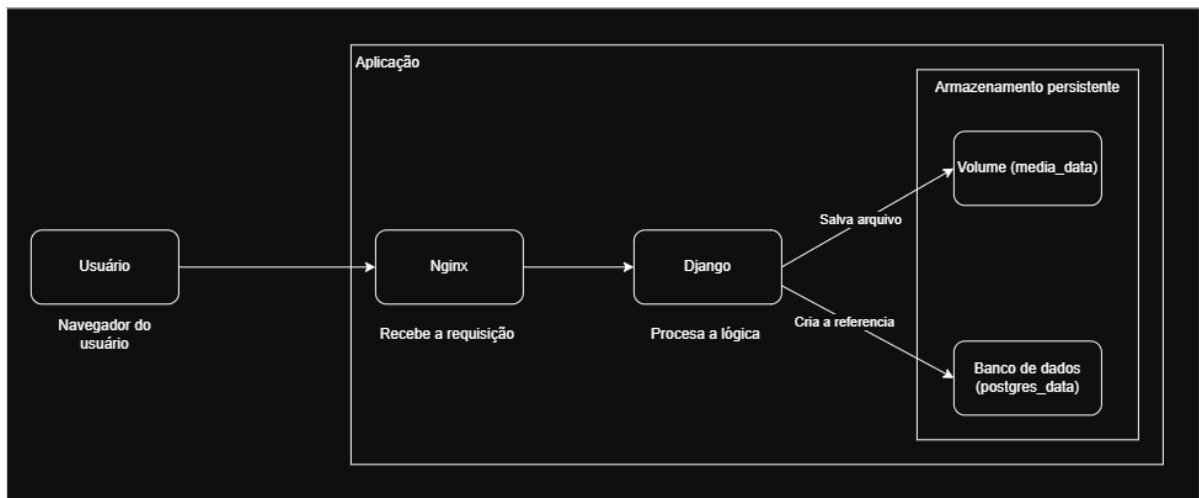
- `listen 80;` e `client_max_body_size 20M;`
Define a porta pública de entrada (80) e limita o tamanho máximo de envio de arquivos/uploads para 20 megabytes.
- `upstream django { server web:8000; }`
Cria o grupo do servidor de destino apontando para o nome do serviço (web) e porta interna (8000), resolvido pelo DNS do Docker.
- `location /static/ e /media/ (alias /app/...)`
Mapeia as URLs de arquivos estáticos e de mídia diretamente para as pastas nos volumes compartilhados, servindo-os sem acionar o Python.
- `proxy_pass http://django;`
Comando principal do proxy reverso; envia as requisições de rotas dinâmicas do sistema para o container do Django/Gunicorn.
- `proxy_set_header Host $host;` e `proxy_set_header X-Real-IP...`
Preserva o cabeçalho original e o IP real do cliente ao repassar a requisição, permitindo que a validação de segurança `CSRF_TRUSTED_ORIGINS` do Django aceite a conexão.

6 - Persistência de Dados

- Solução para Volatilidade: Containers são efêmeros (dados internos somem ao desligar). Usamos Volumes Nomeados para salvar as informações no disco da máquina hospedeira.
- PostgreSQL: O volume `postgres_data` aponta para `/var/lib/postgresql/data`, isolando e salvando todas as tabelas do banco.

- Uploads de Usuários: O volume `media_data` guarda arquivos em `/app/media`. O Django grava (leitura/escrita) e o Nginx acessa em somente leitura (`:ro`).
- Impacto na Destruição/Recriação:
 - Com `docker compose down`, os containers são deletados, mas os volumes permanecem intactos.
 - Ao rodar `docker compose up` novamente, os novos containers reconectam aos volumes com zero perda de dados (a menos que a flag `-v` seja usada para apagar os volumes).

7 - Fluxo do upload de arquivo



8 - Análise técnica da solução

A solução implementa uma arquitetura containerizada em microsserviços/camadas (Nginx, Django/Gunicorn e PostgreSQL), orquestrada via Docker Compose em uma rede isolada (**backend**).

- Fluxo Completo: O cliente acessa o Nginx na porta 80. O Nginx atua como *Gateway* e *Proxy Reverso*: se a requisição for para estáticos ou uploads (`/static/` ou `/media/`), ele responde direto consultando os volumes compartilhados no disco; se for uma rota dinâmica, ele repassa a chamada para o Gunicorn (`web : 8000`). O Django processa a lógica de negócios e consulta o PostgreSQL (`db : 5432`) utilizando o DNS interno da rede do Docker.

Vantagens da Arquitetura Implementada

- Isolamento de Rede e Princípio do Menor Privilégio: Apenas o Nginx expõe uma porta para o mundo exterior (porta 80). O servidor de aplicação (`web`) e o banco de

dados (db) ficam inacessíveis diretamente pela internet, comunicando-se exclusivamente dentro da rede privada do Docker via DNS interno.

- Descarregamento de Carga (*Offloading* de I/O): O Nginx entrega arquivos estáticos e de mídia diretamente dos volumes em disco. Isso impede que o Python/Gunicorn desperdice processos e memória servindo arquivos brutos, mantendo a aplicação leve para responder requisições dinâmicas.
- Infraestrutura como Código (IaC) e Imutabilidade: Toda a stack (dependências do SO, bibliotecas Python, rede, volumes e variáveis) está totalmente declarada nos `Dockerfiles` e `docker-compose.yml`. O ambiente é 100% reproduzível e idêntico em qualquer servidor.

Limitações da Arquitetura Atual

- Ponto Único de Falha (*Single Point of Failure*): Toda a infraestrutura roda em uma única máquina (*Single Host*). Se o servidor físico/VM cair, todos os containers e o banco de dados saem do ar simultaneamente.
- Aplicações Presas ao Disco Local (Impede Escala Horizontal): Os uploads ficam gravados no volume `media_data` da máquina local. Se for necessário rodar 5 containers Django em instâncias diferentes para aguentar mais tráfego, os arquivos enviados em uma máquina não estarão visíveis para as outras.
- Comunicação Insegura (Tráfego em HTTP puro): A aplicação só aceita conexões na porta 80 sem criptografia TLS/SSL. Senhas, uploads e dados trafegam em texto aberto.

Melhorias para Aproximação a um Ambiente de Produção

- Migração para *Object Storage*: Substituir o volume local `media_data` por um serviço de armazenamento de objetos em nuvem usando a biblioteca `django-storages`. Isso permite escalabilidade horizontal ilimitada e alta disponibilidade de mídia.
- Terminação TLS/SSL e HTTPS no Nginx: Adicionar suporte à porta 443 no Nginx com certificados gerados via Let's Encrypt / Certbot, forçando o redirecionamento automático de todo o tráfego de HTTP para HTTPS.
- Banco de Dados Gerenciado (AWS RDS / GCP Cloud SQL) e Orquestrador (Kubernetes): Em produção, o banco de dados não deve rodar em container local, mas em um serviço gerenciado com réplicas e backups automáticos. Além disso, trocar o Docker Compose por um orquestrador como Kubernetes (K8s) ou ECS garantiria *autoscaling* e automação de falhas (*self-healing*).